

Submission Worksheet

IT202-450-M2026 / it202-milestone-1-2026

Submission Data

Course	IT202-450-M2026
Assignment	IT202 Milestone 1 - 2026
Student	Jason T. (jmt86)
Status	submitted
Worksheet Progress	83%
Potential Grade	NaN/NaN (NaN%)
Updated	7/13/2026, 9:42:18 PM

Grading Link

<https://courses.ethereallab.app/assignment/v3/IT202-450-M2026/it202-milestone-1-2026/grading/jmt86>

View Link

<https://courses.ethereallab.app/assignment/v3/IT202-450-M2026/it202-milestone-1-2026/jmt86>

Instructions

Verify that your lesson-built Milestone 1 account system is complete and functional, then collect focused evidence for the account, access, role, profile, and styling requirements. Most of the coding should already be done from the lessons; this worksheet focuses on confirming the required behavior, polishing the project CSS, and explaining your understanding.

Assignment Setup

Required branch: `Milestone1`

Required starting branch: `qa`

Required project folder: `public_html/project/`

Required deployed evidence target: working `qa` site before production promotion

Recommended order:

- Read the whole worksheet.

- Review the Milestone 1 assignment requirements.

- Check out `qa`, pull the latest version, and create the milestone branch.

- Ensure the related lessons are complete in your project.

- Test each feature and requirement.

- Apply your custom CSS to replace the temporary lesson example CSS.

- Merge the milestone branch into `qa`, test deployed behavior, then collect worksheet evidence.

Git Workflow Checklist

- Check out `qa`.

- Pull the latest `qa` branch with `git pull origin qa`.

- Create the milestone branch with `git checkout -b Milestone1`.

- Commit focused verification, fixes, and styling updates in small pieces. If there's nothing to commit, wait until you add the PDF to your repository.

- Push `Milestone1`.

add the PDF to your repository.

Push `Milestone1`.

Open a pull request from `Milestone1` into `qa` and keep it open while you work.

After all Milestone 1 requirements pass locally, merge the pull request into `qa`.

Gather evidence from the deployed `qa` site and fill out the worksheet.

Export the worksheet PDF from the Learn Courses Platform.

Save the PDF in the repository under `docs/milestone01/`.

Add, commit, and push the PDF to the `Milestone1` branch.

```
git add .
git commit -m "Add milestone 1 worksheet PDF"
git push origin Milestone1
```

Upload the same PDF to Canvas.

Create and merge a pull request from `qa` into `prod` when production evidence is required.

Switch back to `qa`.

Pull the latest `qa` branch.

Shared Requirements

Apply these requirements to every Milestone 1 feature:

Add a comment with your UCID, date, and a brief summary of the code block or feature area.

Follow the shared helper, validation, flash message, session, and role-check patterns from the lessons.

Replace the temporary lesson CSS with your own project styling.

Use user-friendly flash messages for success and failure states.

Log technical errors with `error_log()` instead of showing raw technical output to users.

Use HTML validation attributes, JavaScript validation through the page's `validate()` function, and PHP validation before database writes.

Keep safe form values sticky after validation errors.

Clear password fields after validation errors.

Test pages through the supported deployed `qa` URLs before collecting browser evidence.

Use your own code, database behavior, branch, pull request, deployed URLs, and explanation evidence.

Evidence Format

Most sections use the same evidence pattern:

Images:

Show relevant code and matching browser output.

Make sure UCID/date comments are visible in code evidence.

Make sure the deployed `qa` URL is visible in browser output evidence.

One combined screenshot is acceptable when the code and output are both readable.

Use multiple screenshots when one image would be hard to read.

URLs:

Provide direct GitHub file links from the milestone branch.

Provide anticipated production URLs for deployed PHP pages.

Capture the tested `qa` URL, change `-qa` to `-prod`, and keep the same path and filename.

PHP page URLs must end in `.php`; zero credit for that URL entry if it does not.

Response:

Explain the logic in your own words.

Describe the steps or decisions your PHP, SQL, or JavaScript makes.

Do not restate the prompt or list the requirements.

Submission Check

Submission Check

- Confirm the Milestone 1 branch has been pushed.
- Confirm the pull request into `qa` exists.
- Confirm meaningful Milestone 1 commits are visible.
- Confirm the deployed `qa` version was reviewed before promotion.
- Confirm registration, login, logout, protected access, role access, profile edit, and password change were tested.
- Confirm the temporary lesson CSS has been replaced with your own styling.
- Confirm anticipated production URLs use the same path as the tested `qa` URLs with `-qa` changed to `-prod`.
- Confirm the worksheet PDF is committed under `docs/milestone01/` on the milestone branch.
- Upload the same worksheet PDF to Canvas.

Submission Progress

83%

Section #1: (1 pt.) Account Database

100%

Goal: show that the account system has the expected database structure.

Expected database behavior:

- `Users` stores username, email, password hash, created, and modified values.
Username and email are unique.
- `Roles` stores available roles.
- `UserRoles` connects users and roles.

Section #2: (1 pt.) Shared Setup And Helpers

100%

Goal: show that the project loads and uses reusable account helpers from a shared setup file.

Expected helper behavior:

Shared app setup loads the database, session, validation helpers, user helpers, role helpers, flash helpers, and other project utilities.

User helper functions safely check login state before reading user session values.

Role helper functions can check the current user's roles.

Section #3: (2 pts.) Registration

100%

Goal: show that users can register with username, email, password, and confirm password.

Expected behavior:

- HTML, JavaScript, and PHP validation are all present.

- Duplicate username and duplicate email cases show specific, user-friendly feedback.

- Passwords are stored as hashes.

- Safe form values remain sticky after validation errors.

Section #4: (2 pts.) Login And Logout

100%

Goal: show that users can log in by username or email and fully log out.

Expected login behavior:

- Login uses one identifier field.
- The server-side lookup safely checks username or email.
- Passwords are verified with `password_verify()`.
- The session stores the user's safe details and roles.

Expected logout behavior:

- Logout destroys the session.
- Logout redirects with a friendly flash message.
- Reloading a protected page after logout does not allow access.

Section #5: (2 pts.) Profile

100%

Goal: show that users can view and update profile information safely.

Expected account-details behavior

Goal: show that users can view and update profile information safely.

Expected account-details behavior:

- Profile view shows safe account information.

- Username and email can be updated without the current password.

- Duplicate username and email cases show specific, user-friendly feedback.

- HTML, JavaScript, and PHP validation are all present.

Expected password-change behavior:

- Current password is required before saving a new password.

- New password and confirm password are validated.

- Successful password changes store a new hash.

Section #6: (1 pt.) Access Roles And Navigation

50%

Goal: show that access rules are enforced by the server, reflected in navigation, and presented with your own project styling.

Expected behavior:

- Logged-out users cannot access protected pages.

- Users without the required role cannot access role-protected pages.

- Navigation changes based on login state.

- The project uses customized CSS instead of the temporary lesson example styling.

- Role pages from the lessons, such as create/list/assign roles, can satisfy the role-protected workflow.

the 1990s, the number of people in the world who are under 15 years of age is expected to increase from 1.1 billion to 1.5 billion.

As the world's population grows, the demand for food and other resources will increase. This will put pressure on the environment and on the world's food supply.

One way to meet this demand is to increase the amount of food that is produced. This can be done by using more land for agriculture, by using more fertilizers and pesticides, and by using more water.

Another way to meet this demand is to reduce the amount of food that is wasted. This can be done by using less food, by using food more efficiently, and by reducing food losses.

There are many ways to meet the world's growing demand for food and other resources. It is up to us to decide which way is best.

One way to decide is to look at the costs and benefits of each option. This will help us to choose the option that is best for the world.

Another way to decide is to look at the impact of each option on the environment. This will help us to choose the option that is best for the planet.

There are many other ways to decide. It is up to us to choose the way that is best for the world.

One way to choose is to look at the needs of the world's poor. This will help us to choose the option that is best for the most people.

Another way to choose is to look at the needs of the world's future generations. This will help us to choose the option that is best for the world.

There are many other ways to choose. It is up to us to choose the way that is best for the world.

One way to choose is to look at the needs of the world's poor. This will help us to choose the option that is best for the most people.

Another way to choose is to look at the needs of the world's future generations. This will help us to choose the option that is best for the world.

There are many other ways to choose. It is up to us to choose the way that is best for the world.

One way to choose is to look at the needs of the world's poor. This will help us to choose the option that is best for the most people.

Another way to choose is to look at the needs of the world's future generations. This will help us to choose the option that is best for the world.

There are many other ways to choose. It is up to us to choose the way that is best for the world.

Section #7: (1 pt.) Misc



